

Project 1 : Fast LU decomposition

1 Overview

For this project, you will write a function to perform an LU decomposition with partial pivoting on a real-valued square matrix. You will primarily be graded on correctness (does your function return the correct packed factorization and permutation vector, within floating point error?), with the rest of the grade coming from efficiency (is your function as fast as the benchmarks?).

The goals of the project are two-fold. First, you will practice formalizing a linear algebra technique as a concrete algorithm (correctness). Second, you will practice respecting memory layout and hierarchy (efficiency). After this project, I fully expect you to use the pre-built LU decomposition functions (and the functions for other common factorizations) provided by LAPACK (and BLAS, under the hood). However, after this project, you will have an appreciation of *why* LAPACK/BLAS are so fast, and those same techniques will be relevant to any non-standard functions you write in the future.

2 Requirements

1. You must submit a single `.jl` file containing the definition of a function named `lu`. Your file may (and probably should) contain other functions that `lu` calls.
2. Your `.jl` must not import any packages outside of `Base`.
3. Your function must take a real-valued matrix as its **only mandatory** argument, and return a named tuple containing the packed factorization `LU` and the permutation vector `p`, by those names. Your function may have optional arguments with set defaults, but I will not change those values while testing (see Section 3), so make sure they are set at your desired values. Your function skeleton should look like

```
function lu(A::Matrix{T}; [option keyword arguments here]) where T<:Real
    # Function body here
    return (LU = LU, p = p)
end
```

Again, return the factorization **packed into a single matrix**, the way LAPACK's `getrf` does. $\mathbf{L}$ and $\mathbf{U}$ are the two triangles of one $n \times n$ array: for $i > j$, $\text{LU}[i, j]$ is the multiplier belonging to $\mathbf{L}$; for $i \leq j$, $\text{LU}[i, j]$ is the entry of $\mathbf{U}$. The unit diagonal of $\mathbf{L}$ is implicit and is *not* stored — the diagonal of LU belongs to $\mathbf{U}$. Together with `p` this must satisfy

$$\mathbf{LU} = \mathbf{A}[\mathbf{p}, :].$$

Do not build separate $\mathbf{L}$ and $\mathbf{U}$ matrices. Gaussian elimination produces one matrix, not two, and the space where the eliminated entries used to sit is exactly the space the multipliers

need. Storing them separately means allocating twice the memory to write zeros into half of it, and you would be doing that work on every call.

4. Your function must not modify its argument.
5. All floating point operations must be explicitly between scalars; no broadcasting or vectorized operations. Use `for` loops and scalar operations. This may seem tedious, but it is easy to obscure memory layout (and violations thereof) when using vectorized operations, and that is a concept this project is trying to teach. For example, if computing the operation $\mathbf{y} = m \cdot \mathbf{x} + \mathbf{b}$ for vectors $\mathbf{y}, \mathbf{x}$ and scalar m , **do not** write

```
@. y = m*x + b
```

but instead something like

```
for i in eachindex(x)
    y[i] = m*x[i] + b[i]
end
```

This restriction is about *your* arithmetic, not about what the compiler is allowed to do with it. The loop-level annotations `@inbounds` and `@simd` are part of `Base`, are permitted, and are in fact **strongly encouraged**. `@inbounds` elides bounds checking on array accesses within the loop, and `@simd` tells the compiler it may vectorize and reassociate the loop. Neither one vectorizes your code for you — you still write the scalar loop, and you are still responsible for traversing memory in the right order — they simply remove obstacles that prevent the compiler from emitting good code. The example above becomes

```
@inbounds @simd for i in eachindex(x)
    y[i] = m*x[i] + b[i]
end
```

One warning: `@inbounds` disables the bounds check that would normally catch an out-of-range index. Inside an `@inbounds` loop, an indexing bug will silently read or overwrite unrelated memory, or segfault, instead of raising a helpful `BoundsError`. Write and debug your factorization without it, confirm your factorization is correct, and only then add it.

3 Testing

You are given two files, `test.jl` and `project1.sbatch`. Program `test.jl` is how your submission will be tested and `project1.sbatch` is how this test is submitted to the cluster.

3.1 test.jl

Program `test.jl` loads your script as a module, simulates a random $\mathbf{L}$ and $\mathbf{U}$ of size $n \times n$ and forms $\mathbf{A} = \mathbf{L}\mathbf{U}$, calls your `lu`, checks that the factorization is correct, and then times it with `@btime`.

Point the `test.jl` at your submission by editing `FILE_PATH` near the top of `test.jl`:

```
FILE_PATH = "my_sooper_doooper_lu.jl"
module Submission
    include(Main.FILE_PATH)
end
```

You can run `test.jl` with a given dimension n from a command line:

```
julia test.jl -n 500 # Runs the test with n=500
```

An example of a successful output might be:

```
[ Info: Testing my_sooper_doooper_lu.jl
[ Info: Running with n=500
[ Info: Your factorization is correct.
[ Info: Benchmarking...
10.354 ms (6 allocations: 1.91 MiB)
```

You are welcome to play around with your functions outside of `test.jl`, and for development, that is probably wise. But `test.jl` is how I will test and grade your submission. Remember, **do not run anything on the login node of the HPC**.

3.2 project1.sbatch

File `project1.sbatch` submits the above test to a `compute` node on the HPC with a fixed amount of resources. The file is given in full below:

```
#!/bin/bash
# -----
# Project 1
# -----
#SBATCH --job-name=project1
#SBATCH --output=project1-%j.out
#SBATCH --time=00:30:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --partition=compute
#SBATCH --exclusive

module load julia

julia test.jl -n 500
```

Make sure you change the `-n` flag to whatever dimension you'd like to test on the cluster.

The `--exclusive` flag tells SLURM you want a node to yourself. This is necessary if you want to prevent another user from “stepping on your toes” while you are testing. However, it will dramatically increase your queue time. I recommend commenting this out for initial tests:

```
##SBATCH --exclusive
```

If your function beats the benchmark times (See Section 4.2) without an exclusive node, great, feel free to submit. If you are just *barely* missing a benchmark, run it with `--exclusive` and go get lunch while you wait.

4 Grading

You will be graded 90% on correctness, 10% on efficiency, and a possible +10% bonus for extreme efficiency. Everything you earn from those three is then multiplied by whether you can explain your own code, as described in Section 4.3.

4.1 Correctness (90%)

Your function must return the correct packed LU factorization and permutation vector for any n and full-rank matrix A . One way this can be verified is `test.jl` outputting

```
[ Info: Your factorization is correct.
```

The program will stop with an error if your factorization is incorrect. If your function returns the correct values, you will receive a 90% minimum, subject to Section 4.3. As stated in the syllabus, an incorrect algorithm is not eligible for the (10 + 10)% efficiency. You cannot submit a “Hello World!” script and get 20% for your program’s blazing speed.

4.2 Efficiency (10%, plus 10% bonus)

You will receive 10% for efficiency if your function is faster than the baseline, plus a small safety buffer in your favor, at $n = 500$. The baseline algorithm respects Julia’s column-major storage and makes judicious use of the `@inbounds` and `@simd` annotations.

You will receive an additional 10% bonus if your function is faster than the roofline, again plus a small safety buffer in your favor, at $n = 3000$. The roofline utilizes a block-LU algorithm in addition to the techniques in the baseline.

	size	you must beat	my unblocked	my blocked
Baseline	-n 500	12.5 ms	10.3 ms	8.0 ms
Roofline	-n 3000	2.00 s	5.1 s	1.6 s

I will time your functions by submitting `test.jl` through `project1.sbatch`.

4.3 Understanding (all or nothing)

You must understand every line in your code. To ensure this, you will be asked to schedule a 10 minute code review with me. We will look at your code together and I will ask you questions. If you understand the submitted code, you will receive your full grade, as calculated from the categories above. If you do not understand the submitted code, I will give Claude a 110% for the project and you a 0%.

Using LLMs to help you understand material and write code is an excellent idea. But you better be able to defend every line of code that you put your name behind. That is what will be expected of you in industry, and that is what I expect of you here.

5 Submission

Upload the `.jl` file containing your function and its dependencies to D2L. Name the submitted file after yourself, all lower-case, with underscores between your first and last name, e.g. `paul_may.jl`.